

AST Smart Asset

Security Vulnerability & Findings Report

Comprehensive Security Assessment — Confirmed Findings

Assessment Date	September 2026	Report Type	Security Vulnerability & Findings Report
Application	AST Smart Asset Backend API	Status	Confirmed Findings

1. Important Classification Note

During the security assessment, **6 test cases** were classified as **FAIL/FINDING**. However, these represent **3 unique security findings**, because multiple tests identified the same underlying vulnerability.

Failed Test Cases	Unique Security Findings
6	3

Failed Test Case	Finding
Asset Admin accessing an Engineering asset directly	AST-SEC-001
Asset Admin accessing an Engineering asset through the asset list	AST-SEC-001
Asset Admin accessing Engineering custody history	AST-SEC-001
Asset Admin accessing /org-units and seeing out-of-scope org units	AST-SEC-002
Asset Admin accessing /locations and seeing out-of-scope locations	AST-SEC-002
Malformed JSON causing HTTP 500	AST-SEC-003

The failed test cases are not counted as separate vulnerabilities when they represent different manifestations of the same underlying security weakness.

2. AST-SEC-001 – Broken Organizational Authorization / BOLA

AST-SEC-001

Broken Organizational Authorization / BOLA

HIGH

Severity: HIGH

Status: CONFIRMED

Affected Role: asset_admin

Description

The application correctly verifies that the user possesses an authorized **role**, but some endpoints do not additionally verify whether the requested resource belongs to the user's authorized **organizational scope**. This creates a form of **Broken Object Level Authorization (BOLA)** combined with a broken organizational-scope control.

How It Was Discovered

An **asset_admin** user assigned to *Faculty of AI & Data Management* was tested against assets belonging to *Engineering* — which was outside the user's authorized organizational scope. The following requests nevertheless returned successful responses:

- GET /api/v1/assets/:assetId
- GET /api/v1/assets
- GET /api/v1/custody/:assetId/history

Failed Test Cases

Test 1 – Direct Asset Access	
Observed:	HTTP 200 OK with the asset information.
Expected:	Request should be rejected — asset is outside the user's organizational scope.

Test 2 – Asset List Scope Filtering (GET /api/v1/assets)	
Observed:	Response contained assets belonging to Engineering.
Expected:	Response should contain only assets within the user's authorized organizational scope.

Test 3 – Custody History	
Observed:	HTTP 200 OK with custody history data for an out-of-scope asset.
Expected:	Access should be denied — asset is outside the user's organizational scope.

Root Cause

The RBAC middleware checks whether the user has an allowed **role**, but it does not automatically enforce the user's organizational scope. The project already contains the organizational-scope helper **isOrgUnitInUserScope()**, however the affected controllers do not consistently use this function before returning the requested resource.

Current Model	Required Model
User → Has Allowed Role → Access	User → Has Allowed Role → Org Scope Check → Access

Security Impact

An Asset Admin could potentially access asset-related information belonging to another organizational unit, including:

- Asset information & metadata
- Custody history
- Organizationally restricted resources

Recommended Remediation

Every resource-accessing controller should perform an organizational-scope check before returning data:

1. Authenticate the user.
2. Verify the required role.
3. Retrieve the requested resource.
4. Determine the resource's orgUnitId.
5. Call isOrgUnitInUserScope().
6. Return 403 Forbidden if the resource is outside the user's scope.

*For collection/list endpoints, filtering should occur **server-side at the database/query level**, rather than retrieving all records and relying on client-side filtering.*

Verification After Remediation

- Direct Engineering asset → 403 Forbidden
- Engineering asset → excluded from Asset Admin's asset list
- Engineering custody history → 403 Forbidden

3. AST-SEC-002 – Organizational Scope Disclosure / Broken Scope Filtering

AST-SEC-002	Organizational Scope Disclosure / Broken Scope Filtering	MEDIUM
Severity: MEDIUM	Status: CONFIRMED	Affected Role: asset_admin

Description

The organizational hierarchy and location endpoints expose information outside the authenticated user's organizational scope. The Asset Admin assigned to *Faculty of AI & Data Management* could retrieve organizational information related to other units including *Engineering* and *Allied Health*.

Failed Test Cases

Test 4 – Organizational Units (GET /api/v1/org-units)	
Observed:	HTTP 200 OK with the organizational hierarchy, including units outside the user's assigned scope.
Expected:	Only organizational units that the user is authorized to view should be returned.

Test 5 – Locations (GET /api/v1/locations)	
Observed:	HTTP 200 OK with locations associated with organizational units outside the user's scope.
Expected:	Only locations within the user's authorized organizational scope should be returned.

Security Impact

This does not necessarily provide direct access to asset data, but it exposes organizational metadata that may not be intended for the role, including:

- Organizational structure & department relationships
- Location names & internal resource organization
- Identifiers used by other API operations (enabling further authorization attacks)

Root Cause

The affected endpoints appear to return organizational/location records without applying the same organizational-scope filtering used by the application's authorization model.

Recommended Remediation

Apply server-side organizational-scope filtering to:

- GET /api/v1/org-units
- GET /api/v1/locations

If certain global metadata is intentionally visible to authenticated users, this behavior should be explicitly documented as part of the authorization design.

Verification After Remediation

- Asset Admin should not receive unrelated organizational units.
- Asset Admin should not receive unrelated locations.
- Super Admin/global roles may continue to receive the complete hierarchy if that is the intended design.

4. AST-SEC-003 – Improper Error Handling for Malformed JSON

AST-SEC-003	Improper Error Handling for Malformed JSON	LOW
Severity: LOW	Status: CONFIRMED	Affected Endpoint: POST /api/v1/assets

Description

Sending malformed JSON to the API caused the server to return **HTTP 500 Internal Server Error** instead of an appropriate client-side error such as **HTTP 400 Bad Request**.

How It Was Discovered

A deliberately malformed JSON request was sent to the asset creation endpoint.

Test 6 – Malformed JSON	
Observed:	HTTP 500 Error type: INTERNAL_SERVER_ERROR Generic error message (no stack trace or sensitive data exposed).
Expected:	HTTP 400 Bad Request with a stable error code such as INVALID_JSON.

Security Impact

The issue does not currently expose sensitive information — the response contained no stack traces, file paths, database errors, SQL queries, Prisma internals, secrets, or credentials. However, malformed client input should not be treated as an internal server failure. Repeated malformed requests could also create unnecessary server-side error processing and make monitoring/logging less accurate.

Recommended Remediation

Implement centralized malformed-JSON error handling. The API should return:

- HTTP 400 Bad Request
- Stable error code: INVALID_JSON
- Generic message: "Request body contains invalid JSON."

*The server should reserve **500 Internal Server Error** for unexpected server-side failures only.*

Verification After Remediation

- Send malformed JSON → Expected: 400 Bad Request (not 500 Internal Server Error)

5. Overall Findings Summary

Finding ID	Finding	Severity	Failed Tests
AST-SEC-001	Broken Organizational Authorization / BOLA	High	3
AST-SEC-002	Organizational Scope Disclosure / Broken Scope Filtering	Medium	2
AST-SEC-003	Improper Error Handling for Malformed JSON	Low	1
	Total		6 Failed Test Cases

6. Final Assessment

6	3	1	1	1
Failed Test Cases	Unique Findings	High Severity	Medium Severity	Low Severity

The most significant finding is **AST-SEC-001**, where role-based authorization is not consistently combined with organizational-scope authorization. The three failed tests associated with AST-SEC-001 are **not counted as three separate vulnerabilities** — they are separate test cases demonstrating different manifestations of the same underlying authorization weakness.

Similarly, the two organizational disclosure tests are grouped under **AST-SEC-002** because they represent the same underlying scope-filtering problem across two related endpoints. The malformed JSON issue is independent and is therefore recorded as **AST-SEC-003**.

Recommended Remediation Priority

Priority	Finding	Action
1	AST-SEC-001	Implement mandatory organizational-scope authorization checks on resource endpoints and server-side scope filtering on collection endpoints.
2	AST-SEC-002	Apply organizational-scope filtering to organizational hierarchy and location endpoints.
3	AST-SEC-003	Implement centralized malformed-JSON handling and return HTTP 400 Bad Request.

After remediation, perform a focused **re-test** of all six failed test cases to verify that the findings have been successfully resolved.